Versión 1.1

Paradigmas de Programación

Paradigma Lógico

ING. DAVID CECCHI
ING. JACOBO DOMINGUEZ
ING. NICOLÁS TIZZANO

UNIVERSIDAD TECNOLOGICA NACIONAL
Facultad Regional San Nicolás
Ingeniería en Sistemas

El presente documento tiene por objetivo ampliar los conceptos planteados en la Unidad 1 e incorporar nuevos, haciendo hincapié en el uso del lenguaje Prolog a través de ejemplos prácticos.

Paradigma Lógico

En el paradigma lógico, un **programa** se expresa como un conjunto de hechos (declaraciones que son verdaderas dentro del dominio del problema) y reglas (relaciones lógicas entre esos hechos e incluso con otras reglas previamente definidas) escritas con fórmulas lógicas de primer orden. Dicho conjunto constituye la base de conocimientos del problema que se desea afrontar. El objetivo de la ejecución será intentar demostrar la veracidad, dentro de esa teoría (dominio o base de conocimientos), de consultas realizadas.
En consecuencia, el primer paso para resolver un problema implementando el paradigma lógico, es generar la base de conocimiento sobre la cual se realizarán las consultas de interés.
La lógica de primer orden que admite el paradigma se limita a las Cláusulas de Horn, un subconjunto restringido de la lógica de primer orden, evitando así enfrentar el "problema de decisión" planteado por Hilbert el cual hacía referencia a toda la lógica de primer orden y que Turing y Church demostraron su indecibilidad.
En el subconjunto de la lógica de primer orden donde se desenvuelve el paradigma lógico se habla de éxito o fracaso (falla), no así de verdadero o falso. Se entiende por éxito cuando la consulta es verificable a partir de la base de conocimiento existente, y se considera falla cuando a partir del dominio la consulta no se verifica o no puede ser demostrada.
Se selecciona al lenguaje Prolog para la implementación del paradigma. Con el objetivo de facilitar la interpretación de conceptos se recurrirá a ejemplos prácticos, necesitando para ello iniciar con el detalle de los tipos de datos, operadores y sintaxis del lenguaje.

Cláusulas de Horn
Es el subconjunto restringido de lógica de primer orden que admite el paradigma, esta restricción lo mantiene dentro del espacio de lo decidible.
Las restricciones, con respecto a la lógica de primer orden, son:

- No se permiten conclusiones múltiples: A or B ni A and B.
- No se permiten negaciones en la cabecera (conclusiones).
- No se permiten disyunciones (OR) en las condiciones, si es necesario, se deberán expresar dos reglas separadas.
- Los cuantificadores “Para Todo” y “Existe” se encuentran implícitos y restringidos.

Tipos de Datos

Tipos Simples	
Átomos	Constantes simbólicas, comienzan con minúsculas. Para utilizarlos con espacios o mayúsculas deben encerrarse entre comillas simples. Ejemplo: ana, camello.
Números	Entero o punto flotante. Ejemplo: 12, 3.54.
Cadenas	Deben encerrarse entre comillas dobles (alt+34).
Tipos Compuestos	
Estructuras	Un nombre y un conjunto de elementos, delimitados por coma y encerrados entre paréntesis. Ejemplo: fecha(1990,3,17)
Listas	Conjunto de elementos delimitados por coma y encerrados entre corchetes. Ejemplo: [], [12, 54, 170], [ana, luis, juan].

Estructuras

Una estructura permite agrupar información relacionada bajo un único nombre. Los elementos agrupados se encierran entre paréntesis y se encuentra separados por comas.
Puede combinar elementos heterogéneos, es decir: átomos, estructuras, cadenas, listas, etc.

- producto(manzana, verde, 3250.70).
- alumno(gomez, fecha(2026, 3, 10), "Bolivar 370", isi).

Listas

Una lista es una serie de elementos, los cuales pueden ser de diferentes tipos. Los elementos se encierran entre corchetes y se encuentran separados por comas. Se admite las listas de listas. Ejemplos:

- [21, 17, 34]
- ["Martillo", "Pinza", "Destronillador"]
- [21, "Martillo de 1kg", 17.5, "Dewalt"]
- [[21, 17, 34], [13,14], [18]]

Patrón	Descripción
[]	Lista vacía
[X]	Lista de un solo elemento
[X|XS]	El operador | separa la cabeza de la cola de la lista.

Algunas Reglas Sintácticas

- Los átomos comienzan con una minúscula, si contiene espacios debe encerrarse entre comillas simples.
- Las variables comienzan con una mayúscula o un guion bajo seguido de minúscula.
- Toda clausula finaliza con un punto.
- Para comentar una línea se utiliza el símbolo % al inicio de ella.
- Para comentar múltiples líneas, el bloque debe encerrarse entre /* y */.

Operadores Aritméticos

Operador	Significado
+	Suma
-	Resta
*	Producto
/	División real
//	División entera
mod	Resto (módulo)
**	Potencia
abs(n)	Valor Absoluto
max(n, w)	Máximo
min(n, w)	Mínimo
sqrt(n, w)	Raíz Cuadrada

Conceptos

Variable

Una variable representa un término desconocido dentro una cláusula o consulta. Su nombre debe comenzar con una mayúscula o guión bajo. Una vez que una variable es ligada a un término (un átomo, un número, una estructura, una lista u otra variable) mediante la unificación, no puede reasignarse a otro término durante esa misma ejecución: no admite modificación (inmutabilidad). Solo el backtracking deshace la ligadura y la deja libre otra vez.
Ejemplo: X, Cant, _peras.

Unificación

Es el proceso mediante el cual, el sistema liga (o vincula) variables a términos para lograr que ambos términos sean idénticos, es decir, busca una sustitución que los haga iguales. Si no existe dicha sustitución, la unificación falla. Es un mecanismo fundamental del motor de inferencia.
Si uno de los términos a unificar es una variable libre (aún no ligada), la unificación tendrá éxito ya que la variable quedará ligada al otro término. Si ambos términos son variables libres, las mismas quedarán ligadas entre sí: cualquier ligadura posterior de una afecta a la otra.
Se dice que dos términos unifican si se cumplen alguna de las siguientes condiciones:

- Son términos idénticos.
- Existe una sustitución (un término de reemplazo para cada variable) que, aplicada a ambos términos a unificar, los vuelve idénticos entre sí. Cada variable recibe un único término de reemplazo, que se usa en todas sus apariciones.

1. `X = ana.` %la variable X queda ligada a ana, la unificación tiene éxito
2. `X = ana, X = juan.` % El primer objetivo liga X a ana. El segundo objetivo intenta unificar X (ligada a ana) con juan; como ana y juan son átomos diferentes (dos constantes distintas nunca unifican), la unificación falla y, en consecuencia, la consulta completa falla.
3. `X = 2 + 1, X = 3.` % El primer objetivo liga X al término compuesto 2 + 1 (sin evaluar). El segundo objetivo intenta unificar ese compuesto con el entero 3 (un entero nunca unifica con un compuesto); la unificación falla y, en consecuencia, la consulta completa falla.
4. `X is 2 + 1, X = 3.` % En el primer objetivo, is evalúa la expresión aritmética de su derecha y liga X al entero 3. El segundo objetivo intenta unificar X con el entero 3; como X ya está ligada a ese mismo entero, la unificación tiene éxito y la consulta completa tiene éxito.
5. `X = 2 + 1, X == 3.` % El primer objetivo liga X al término compuesto 2 + 1 (sin evaluar). El segundo objetivo compara sus operandos tal como se encuentran (== nunca liga ni evalúa, solo comprueba que sean idénticos). Como el término compuesto ligado a X no es idéntico al entero 3, la comparación falla y, en consecuencia, la consulta completa falla.
6. `X = 2 + 1, X =:= 3.` % El primer objetivo liga X al término compuesto 2 + 1 (sin evaluar). El segundo objetivo, con =:=, evalúa ambos operandos y compara sus valores: el término ligado a X vale 3, al igual que el entero 3, por lo que la comparación tiene éxito y la consulta completa tiene éxito. X sigue ligada a 2 + 1.
7. `X \= 3.` % La variable X, que se encuentra libre, puede ser ligada al entero 3; es decir, la unificación es posible y, en consecuencia, la consulta falla (el operador \= no evalúa, intenta realizar la unificación; si es posible, falla y deshace la ligadura: X permanece libre; en caso contrario, tiene éxito).
8. `X = 3, Y = 2 + 1, X \= Y.` % El primer objetivo liga X al entero 3 y el segundo liga Y al término compuesto 2 + 1 (sin evaluar). El tercer objetivo, con \=, intenta unificar 3 con 2 + 1; un entero nunca unifica con un compuesto, aunque ambos tengan el mismo valor aritmético, porque la unificación compara estructura y no valor. Como la unificación no es posible, \= tiene éxito y la consulta completa tiene éxito.

Terminología utilizada:

- **Consulta**: secuencia de uno o más objetivos planteados por el usuario, ej.: *X = 2 + 1, X == 3*.
- **Objetivo**: cada parte separada por comas que en conjunto constituyen la consulta, ej.: *X = 2 + 1* y *X == 3*.
- **Operandos** (o **términos a unificar**): aquello que está a cada lado del operador que compone a un objetivo, ej.: *X* y *2 + 1*.
- **Término de reemplazo**: es el término al que queda ligada una variable como resultado de una unificación exitosa.

Operador	Evalúa	Unifica	Compara
=	No	Sí	No de forma directa: el resultado (éxito o fallo) informa si los operandos son unificables.
\=	No	Intenta (sin conservar las ligaduras)	No de forma directa: el resultado (éxito o fallo) informa si los operandos no son unificables.
is	Sí, solo operando derecho.	Sí, resultado con operando izquierdo.	No.
==	No	No	Sí, comparación estructural (igualdad).
\==	No	No	Sí, comparación estructural (diferencia).
=:=	Sí, ambos operandos.	No	Sí, comparación aritmética (igualdad).
=\=	Sí, ambos operandos.	No	Sí, comparación aritmética (diferencia).
>, <, =<, >=	Sí, ambos operandos.	No	Sí, comparación aritmética (orden).

Ejercicio 1: se asume que las variables se encuentran libres (no instanciadas).
Objetivo: visualizar la diferencia en el funcionamiento de los operadores = y \=.

1. persona(juan) = persona(luis). % retornará false
2. persona(juan) = persona(P). % retornará true, ya que la unificación instanciando a P como juan hace ambos términos idénticos

3. persona(juan) \= persona(luis). % retornará true
4. persona(juan) \= persona(P). % retornará false, ya que la unificación instanciando a P como juan hace ambos términos idénticos

Ejercicio 2: dado un usuario y recurso, informar si el usuario es el propietario del mismo.
Objetivo: visualizar las consecuencias de no comprender el funcionamiento de los operadores = y ==.

Hechos

```
dueno_de(documento, admin).
dueno_de(fotos, juan).
dueno_de(tesis, ana).
```

Reglas

```
% Versión con unificación(inseguro)
es_dueno_unificacion(Usuario, Recurso) :- dueno_de(Recurso, Propietario), Usuario = Propietario.

% Versión con identidad estricta (seguro)
es_dueno_identidad(Usuario, Recurso) :- dueno_de(Recurso, Propietario), Usuario == Propietario.
```

Consultas

```
% al pasar ambos argumentos no se perciben diferencias
es_dueno_unificacion(juan, fotos). %retorna true
es_dueno_identidad(juan, fotos). %retorna true

% al pasar variables se perciben las diferencias
es_dueno_unificacion(X, tesis). %retorna true, con X = Ana, permite conocer (pescar) el usuario
es_dueno_identidad(X, tesis). %retorna false, mantiene oculto al usuario evitando operar con variables
```

Conclusión: la unificación propia del operador =, en ocasiones, puede generar inconvenientes de seguridad exponiendo información.

Ejercicio 3: dado dos números, informar si el segundo es el siguiente del primero.
Objetivo: visualizar las consecuencias de no comprender el funcionamiento de los operadores = e is.

Reglas

```
% Versión sin evaluación
siguienteIgual(Num, Prox) :- Prox = Num + 1.

% Versión con evaluación
siguienteIs(Num, Prox) :- Prox is Num + 1.
```

Consultas

```
% pasando ambos argumentos para verificar si es el siguiente
siguienteIgual(2, 3). %retorna false, por intentar unificar 3 = 2 + 1 sin haber evaluado
siguienteIs(2, 3). %retorna true, evalúa 2 + 1 y luego intenta unificar.

% al pasar una variable para conocer el siguiente
siguienteIgual(2, N). %retorna true, con N = 2 + 1, sin haber evaluado
siguienteIs(2, N). %retorna true, con N = 3
```

Conclusión: operador =, al no evaluar la expresión, no permite obtener los resultados esperados.

Cláusula

Una cláusula es la unidad sintáctica fundamental, puede ser un hecho o regla.
Cada cláusula debe respetar la estructura sintáctica planteada por algún predicado.
Las cláusulas que se correspondan a un mismo predicado deben ubicarse juntas, una debajo de otra, para mejorar el desempeño del motor de búsqueda. En caso de no cumplir con este consejo, Prolog emitirá una advertencia.

Hecho

Es una declaración concreta que se asume verdadera en el dominio del problema. No incluye condiciones ni variables.

Ejemplos

```
1. humano(juan)        % Juan es humano
```

```
2. padre(juan, pedro)   % Juan es padre de Pedro
3. jefe(lucas, andrea)  % Lucas es jefe de Andrea
4. mascota(toby)        % Toby es una mascota
```

Regla

Una regla es una cláusula condicional de la forma Cabeza :- Cuerpo. La cabeza es el término que expresa una *conclusión*, y el cuerpo es una conjunción de una o más *condiciones* que deben cumplirse para que esa conclusión se considere verdadera. La cabeza determina a qué predicado pertenece la regla.
En Prolog, las reglas se escriben desde la conclusión hacia las condiciones, porque el motor de inferencia trabaja hacia atrás: parte del objetivo que quiere demostrar y busca las condiciones que lo satisfacen.

En lenguaje natural:
Si un Ser es humano, entonces es mortal.

En Prolog:

```
mortal(Ser) :- humano(Ser)
```

Interpretación desde Prolog:
Un Ser es mortal si el Ser es humano.

Ejemplos

```
1. progenitor(X, Y) :- padre(X, Y).          % X es progenitor de Y si X es padre de Y
2. abuelo(X, Z) :- padre(X, Y), padre(Y, Z) . % X es abuelo de Z si X es padre de Y, e Y es padre de Z
3. mortal(X) :- humano(X).                   % X es mortal si X es humano o X es animal
   mortal(X) :- animal(X).
4. hermano(X,Y) :- progenitor(Z,X), progenitor(Z,Y), X \= Y. % X e Y son hermanos si tienen el mismo
                                                              progenitor y X e Y no son la misma persona
```

- Para expresar el operador AND se recurre a la coma.
- Para expresar el operador OR comúnmente se recurre a cláusulas separadas. Es posible utilizar el operador punto y coma, pero algunos intérpretes pese a verificar el cumplimiento de la primera condición continúan evaluando las restantes, este proceso puede arrojar resultados confusos.

Consultas (queries)

Es la secuencia de uno o más objetivos que el usuario plantea para que el sistema la demuestre. El motor de inferencia buscará determinar si puede demostrarse a partir de los hechos y reglas de la base de conocimiento. Por principio de universo cerrado, todo lo que no forma parte de la base de conocimientos se presume falso. En consecuencia, ante una consulta para la cual no se cuenta con el conocimiento necesario, Prolog no señalará "*desconocimiento*" sino que retornará falso.
Por este motivo, no tiene sentido plantear hechos del tipo "Pedro no es docente", "Luis no es padre de Ana", "Andrea no trabaja en San Nicolás", etc. ya que con el solo hecho de no figurar en la base de conocimiento automáticamente, las siguientes consultas se asumen como falsedades: "Pedro es docente", "Luis es padre de Ana", "Andrea trabaja en San Nicolás". No tiene sentido definir aquello que no es.

Ejemplos

1. En la consulta padre(juan, X), X es una variable que el sistema intentará satisfacer. Retornando la lista de personas para las que juan es padre según los registros que figuran en la base de conocimiento.
2. En la consulta padre(_, pedro), _ es una **variable anónima** que el sistema intentará satisfacer. Retornando true si en la base de conocimiento se encuentra al menos una cláusula que cumpla con la consulta.

Functor
Un functor describe a través de una nombre (un átomo) y una aridad (cantidad de argumentos), la estructura de un término compuesto (cláusula, estructura, etc). Se escribe como nombre/aridad.
Por ejemplo, el functor que describe al hecho padre(luis, ana) es padre/2, el functor que describe la conclusión de la regla abuelo(X, Z) es abuelo/2, el functor que describe a la estructura auto(ford, 2026, venta(2026, 4, 23), 35000) es auto/4.

Predicado
Un predicado es el conjunto de cláusulas (hechos y reglas) que comparten un mismo functor (nombre y aridad). Representa una relación entre sus argumentos y se identifica por nombre/aridad.
Si dos conjuntos de cláusulas tienen el mismo nombre pero diferente aridad, se consideran predicados distintos.

Ejemplo
Hechos
```
padre(luis, ana).
padre(juan, maria).
padre(juan, pedro).
padre(fernando, maria, 2017). %podria expresar que fernando es padrastro de maria desde 2017
padre(fernando, pedro, 2017). %podria expresar que fernando es padrastro de pedro desde 2017
```

Predicados
```
padre/2 %representa a los 3 primeros hechos
padre/3 %representa a los 2 últimos hechos
```

Inversibilidad
Es la capacidad de un predicado de funcionar correctamente con distintos modos de uso, es decir, con distintos argumentos ligados o libres, y no solo en la dirección “natural”. La inversibilidad es una consecuencia directa de la unificación.
No todo predicado es inversible. La inversibilidad se ve limitada cuando el predicado incorpora operaciones que exigen argumentos instanciados, como la aritmética con is o las comparaciones numéricas.

Ejemplo con Predicado Inversible
Hechos
```
padre(luis, ana).
padre(juan, maria).
padre(juan, pedro).
padre(javier, juan).
padre(sergio, lucia).
```

Reglas
```
abuelo(X, Y) :- padre(X, Z), padre(Z, Y).
```

Consultas
```
padre(juan, X). %retorna los hijos de juan
padre(X, maria). %retorna los padres de maria
padre(X, Y). %retorna todos los padres con sus hijos
abuelo(javier, N). %retorna los nietos de Javier
abuelo(A, pedro). %retorna el abuelo de pedro
```

El predicado Padre es inversible por lo cual admite diferentes usos.

El predicado Abuelo es inversible por lo cual admite diferentes usos.

Ejemplo con Predicado No Inversible
Reglas

```
suma(X, Y, Z) :- Z is X + Y.
```

Consultas

```
suma(2, 3, Z). %retorna 5
suma(2, Y, 5). %no resuelve, error de instanciación
suma(X, 3, 5). %no resuelve, error de instanciación
```

El predicado suma no es inversible ya que el operador is requiere que el término de la derecha este completamente instanciado.

Pattern Matching

Es el mecanismo que, al evaluar una consulta, compara los argumentos recibidos con los patrones definidos, de arriba hacia abajo.

Al igual que en la programación funcional, el concepto pattern matching también está presente en la programación lógica pero lo hace de diferente manera.

En programación funcional, este mecanismo es unidireccional y determinista (resultado único); se toma el primer patrón que encaje con la expresión y los siguientes patrones no son evaluados.

En programación lógica es bidireccional (las variables pueden instanciarse en cualquier dirección) y no determinista; se dará respuesta explorando mediante el backtracking todos los patrones que unifiquen (coincidan) con la consulta dada entregando las soluciones una a una.

Ejemplo en Programación Funcional

```
describe :: Int -> String
describe 0 = "es cero"
describe n = "es otro número" -- este patrón también matchea 0 pero con cero nunca se llegará a evaluar

main :: IO()
main = print (describe 0) -- retorna “es cero”
```

Ejemplo en Programación Lógica
Hechos

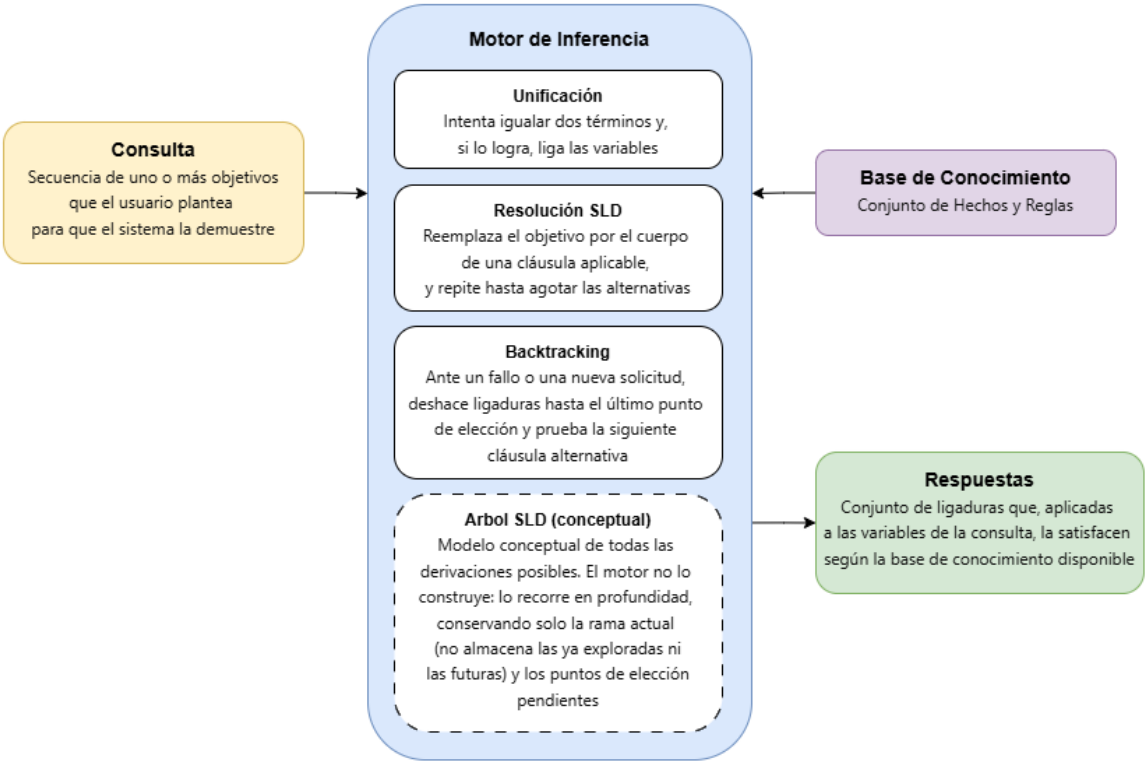
```
describe(0, "es cero").
describe(_, "es otro número"). % también unifica con 0
```

Consulta

```
describe(0, X).
X = "es cero". % primera solución
X = "es otro número". % segunda solución
```

Motor de inferencia

Es el mecanismo que resuelve las consultas: busca demostrar que una consulta se deduce de los hechos y reglas definidas en la base de conocimiento y, si lo logra, obtiene las ligaduras de sus variables. Para ello aplica la **resolución SLD**, cuyos pasos pueden representarse como un árbol de derivaciones (**árbol SLD**), que el motor recorre en profundidad (**DFS**) con **backtracking**, sin construirlo en memoria.



Resolución SLD
Algoritmo de inferencia que intenta demostrar una consulta (una secuencia de objetivos) mediante una serie de pasos: selecciona un objetivo (en Prolog, el más a la izquierda), busca una cláusula cuya cabeza unifique con él y lo reemplaza por el cuerpo de esa cláusula, aplicando las ligaduras obtenidas al resto de los objetivos. Repite este proceso hasta que no quedan objetivos pendientes (éxito) o ninguna cláusula es aplicable (la derivación falla). Si varias cláusulas son aplicables, cada una origina una derivación alternativa, que corresponde a una nueva rama del árbol SLD.

DFS (Depth First Search)
Estrategia de recorrido que explora cada rama hasta el final (hasta el nodo más profundo) antes de pasar a otra. Prolog la aplica probando las cláusulas de arriba hacia abajo y los objetivos de izquierda a derecha.

Árbol SLD
Modelo conceptual de todas las derivaciones posibles de una consulta. Cada nodo es la secuencia de objetivos que aún quedan por demostrar (con las ligaduras acumuladas); cada arista es un paso de resolución, es decir, la aplicación de una cláusula cuya cabeza unifica con el objetivo seleccionado; y cada rama es una derivación completa. Las bifurcaciones aparecen cuando varias cláusulas son aplicables, dando lugar a ramas alternativas. El motor no lo construye en memoria: lo recorre, conservando solo la rama actual y los puntos de elección pendientes.

Backtracking (retroceso)
Mecanismo que permite volver a una decisión anterior para probar otra alternativa. Se activa cuando un objetivo falla o cuando se piden más soluciones: el motor deshace las ligaduras realizadas desde el último punto de elección con cláusulas aún sin probar y continúa con la siguiente. Puede retroceder varios pasos si los intermedios no tienen alternativas pendientes. Permite hallar todas las soluciones, salvo que existan ramas infinitas o se pode el árbol con el corte.

En CVG se adjunta video explicativo sobre el proceso de resolución.

Operador ! (Cut)

El corte (cut, !) es el mecanismo por el cual el programador puede intervenir en la estrategia de búsqueda alterando el backtracking: al ejecutarse, descarta las cláusulas posteriores del predicado al que pertenece la cláusula que lo contiene y las opciones alternativas de los objetivos ubicados a su izquierda. De este modo, el motor deja de retroceder a esas opciones y la búsqueda exhaustiva se transforma en una decisión determinista. Los objetivos que siguen a la derecha del corte, dentro de la misma cláusula, se ejecutan con normalidad.

El corte solo actúa cuando se ejecuta: si la cabeza no unifica o algún objetivo anterior al corte falla, el motor continúa con la cláusula siguiente.

Ejemplo 1

```
clasificar(X, pequeño) :- X < 10, !.
clasificar(X, mediano) :- X < 100, !.
clasificar(_, grande).

clasificar(5, T).     % T = pequeño: se ejecuta el corte y se descartan las dos cláusulas siguientes
clasificar(50, T).    % T = mediano: X < 10 falla, el primer corte no se alcanza, la segunda cláusula tiene éxito
                      y luego el corte descarta la 3er cláusula
clasificar(500, T).   % T = grande: fallan las dos cláusulas anteriores antes de llegar a un corte
```

Ejemplo 2

```
cobro(Edad, Precio) :- Edad < 12, !, Precio = 10.
cobro(Edad, Precio) :- Edad >= 65, !, Precio = 80.
cobro(_, 120).

cobro(5, P).          % P = 10: se ejecuta el corte y se descartan las dos cláusulas siguientes
cobro(30, P).         % P = 120: fallan las dos cláusulas anteriores antes de llegar a un corte
```

Ejemplo 3

Hechos

```
alumno(luis, 2025, estudiante, 20, 7.5).
alumno(andrea, 2026, estudiante, 31, 8.0).
alumno(luisa, 2025, graduado, 25, 9.0).
alumno(juan, 2026, graduado, 30, 6.5).
alumno(fernando, 2026, graduado, 29, 9.5).
alumno(luciana, 2026, graduado, 26, 7.0).
```

Reglas

```
todos_graduados_con_honores(Nom) :-  alumno(Nom, _, graduado, _, Nota), Nota >= 8.
un_graduado_con_honor(Nom) :- alumno(Nom, _, graduado, _, Nota), Nota >= 8, !.

% Se busca el primero con nota >= 9, si existe, no se continúan evaluando las siguientes reglas.
% En caso contrario, se buscarán todos los que cumplan las reglas inferiores.
asignar_beca(Nom, 'Beca Total') :- alumno(Nom, _, _, _, Nota), Nota >= 9, !.
asignar_beca(Nom, 'Beca Parcial') :- alumno(Nom, _, _, _, Nota), Nota >= 7.
asignar_beca(_, 'Sin Beca').
```

Consultas

```
%retorna juan, 30; Fernando, 29
1. alumno(N,2026,graduado,E,P), E>28.

%retorna juan, 30
```

2. alumno(N,2026,graduado,E,P), E>28, !.

   %retorna todos los que cumplen con la regla: luisa y fernando
3. todos_graduados_con_honores(Nombre).

   %retorna el primero que cumple con la regla: luisa
4. un_graduado_con_honor(Nombre).

   %retorna: luisa, Beca Total. Primero que cumple con la primera regla.
5. asignar_beca(Nombre, Decision).

Predicados

Hasta el momento, al predicado se lo ha descripto por su composición: el conjunto de cláusulas (hechos y reglas) que comparten un mismo functor. A continuación, se lo clasificará por su comportamiento, según el uso que hace de sus argumentos: predicados de primer orden y predicados de orden superior.
Un predicado de primer orden es aquel cuyos argumentos son datos: términos (átomos, números, listas, estructuras o variables) que se consultan, se unifican o se construyen, pero no se ejecutan.
Un predicado de orden superior es aquel que recibe como argumento un término, lo interpreta como un objetivo y lo ejecuta. Basta con que uno de sus argumentos se use de este modo.

Predicados de Primer Orden

Predicado	Descripción
length(XS,N)	Retorna en N la longitud de la lista o retorna una lista XS de tamaño N o valida que la lista XS tenga la longitud N.
member(N,XS)	Retorna true si el elemento N se encuentra en la lista XS o los elementos de la lista XS si no se especifica N.
append(XS,LS,YS)	Retorna en YS una lista con la unión de los elementos de las listas XS y LS. Dado las listas XS e YS retornará en LS la lista requerida para conformar YS partiendo de XS. Dado las listas LS e YS retornará en XS la lista requerida para conformar YS partiendo de LS.
nth0(P,XS,E)	Retorna el elemento E que se encuentra en la posición P de la lista XS, tomando la posición 0 como inicio de la lista. Dada la lista XS y el elemento E, retornará la posición P en que se encuentra.
last(XS, E)	Retorna en E el último elemento de la lista XS. Dado XS y E, validara que el elemento E sea el último de la lista.
sum_list(XS,M)	Retorna en M la suma total de los elementos de la lista.
max_member(M,XS)	Retorna en M el mayor elemento de la lista XS (criterio alfanumérico).
min_member(M,XS)	Retorna en M el menor elemento alfanumérico de la lista XS (criterio alfanumérico).
max_list(XS,M)	Retorna en M el mayor elemento de la lista XS (criterio numérico).
min_list(XS,M)	Retorna en M el menor elemento de la lista XS (criterio numérico).

Ejemplos

```
%Retorna true
1. member(45, [18,45,32]).

%Retorna en E el elemento 32
2. nth0(2, [18,45,32], E).

%Retorna en E el elemento 45
3. max_list([18,45,32], E).

%Retorna en T el elemento 95
4. sum_list([18,45,32], T).

%Retorna en L la lista [18,45,32,56,70]
5. append([18,45,32], [56,70], L).

%Se define el hecho Head. La consulta retorna en Primero el elemento 18
6. head([Cabeza | _], Cabeza).
   head([18,45,32], Primero).
```

Predicados de Orden Superior

Predicado	Descripción
findall(F, T, XS)	Retorna en XS la lista de elementos con formato F según los datos que surgen de ejecutar término T.
aggregate_all(A,T,Z)	Retorna en Z el resultado de aplicar el *operador de agregación* A sobre los datos que surgen de ejecutar el término T.

Operador de Agregación	Descripción	Ejemplo
count	Cuenta las soluciones del objetivo	aggregate_all(count, importe(_, _), N)
sum(Expr)	Suma los valores de la expresión	aggregate_all(sum(I), importe(_, I), S)
max(Expr)	Mayor valor de la expresión. Si no hay solución falla.	aggregate_all(max(I), importe(_, I), M)
min(Expr)	Menor valor de la expresión. Si no hay solución falla.	aggregate_all(min(I), importe(_, I), M)
max(Expr, Testigo)	Mayor valor, junto con el testigo que lo alcanza.	aggregate_all(max(I, Id), importe(Id, I), M)
min(Expr, Testigo)	Menor valor, junto con su testigo que lo alcanza.	aggregate_all(min(I, Id), importe(Id, I), M)
bag(X)	Lista de los valores en el orden en que aparecen, **con** repeticiones. Si no hay solución retorna [].	aggregate_all(bag(P), venta(_, _, producto(P, _), _), B)
set(X)	Lista ordenada y **sin** repeticiones. Si no hay solución retorna [].	aggregate_all(set(P), venta(_, _, producto(P, _), _), S)

maplist(T,LS,XS)	Retorna en XS la lista de elementos que surgen al ejecutar el término T a cada elemento de la lista LS.
include(T,LS,XS)	Retorna en XS la lista de elementos de la lista LS que unifican con el término T.
not(T)	Retorna true (éxito) si la ejecución del término T falla (porque no se cumple o no puede ser demostrado a partir del dominio actual).
forall(TC, TA)	Retorna true (éxito) si todos los individuos que cumplen el término TC, también cumplen el término TA.

Ejemplos

Hechos

```
auto(ford, 2026, venta(2026, 4, 23), 35000).
auto(renault, 2021, venta(2022, 3, 4), 17000).
auto(ford, 2023, venta(2024, 10, 7), 22000).
auto(vw, 2025, venta(2025, 8, 10), 20000).
auto(vw, 2025, venta(2026, 1, 14), 25000).
auto(peugeot, 2026, venta(2026, 3, 21), 27000).
```

Reglas

```
duplicar_precio(auto(M, A, V, P1), auto(M, A, V, P2)) :- P2 is P1 * 2.
es_costoso(auto(_, _, _, Precio)) :- Precio > 25000.
```

Consultas

```
%retorna Car: ford y Peugeot
1. auto(Car,2026,venta(2026,_,_),_).

%retorna los autos con precio > 25000
2. findall(auto(M,A,V,P), (auto(M,A,V,P), P>25000), Resultado).

%retorna la cantidad total (2) de autos ford
3. aggregate_all(count, auto(ford,_,venta(_,_,_),_), Cant). %retorna Cant: 2

%retorna los autos con el precio dublicado
4. findall(auto(M, A, V, P), auto(M, A, V, P), ListaOriginal), maplist(duplicar_precio, ListaOriginal,
   Resultado).

%retorna los autos con precio alto
5. findall(auto(M, A, V, P), auto(M, A, V, P), ListaOriginal), include(es_costoso, ListaOriginal, Resultado).

%retorna true porque todos los autos marca Ford tienen un precio > 21000
6. forall(auto(ford,_,_,P), P > 21000)

%retorna false porque no todos los autos marca Ford son considerados costosos
7. forall((auto(M,A,V,P), M==ford), es_costoso(auto(M,A,V,P))).
```

Recursividad

La recursividad es un mecanismo por el cual un predicado se define a partir de sí mismo: alguna de sus cláusulas contiene en su cuerpo un objetivo que invoca al mismo predicado (es decir, con el mismo functor que su cabeza). Debido a que Prolog no cuenta con estructuras de control repetitivas como el for o el while, la recursividad es el recurso principal para expresar procesos repetitivos (otros recursos pueden ser los predicados de orden superior: forall, maplist, etc.).

Una definición recursiva en Prolog siempre debe contar con dos componentes:

- **Caso base:** una cláusula que resuelve el problema para el caso más simple, **sin** llamarse a sí misma. Permite detener la recursión.
- **Caso recursivo:** una cláusula que reduce el problema a una versión simplificada llamándose a sí misma. Debe acercarse al caso base en cada paso.

Ejemplo Longitud de Lista

```
% Caso base: longitud de una lista vacía es 0
longitud([], 0).
% Caso recursivo: longitud de una lista no vacía
longitud([_|Cola], L) :- longitud(Cola, L1), L is L1 + 1.
```

Consulta

```
longitud([23,17,45,82], L)
```

Camino de Resolución

```
longitud ([23,17,45,82], L)
    └─ longitud ([23,17,45], L1)
        └─ longitud ([23,17], L2)
            └─ longitud ([23], L3)
                └─ longitud ([], L4)
                    └─ L4 = 0   ← caso base
                L3 = 0 + 1 = 1
            L2 = 1 + 1 = 2
        L3 = 2 + 1 = 3
L = 3 + 1 = 4
```

Ejemplo Factorial con el caso Base como un Hecho

```
% Caso base (como hecho): el factorial de 0 es 1
factorial(0, 1).

% Caso recursivo: N! = N * (N-1)!
factorial(N, F) :- N > 0, N1 is N - 1, factorial(N1, F1), F is N * F1.
```

Consulta

```
factorial(4, F)
```

Camino de Resolución

```
factorial(4, F)
    └─ factorial(3, F1)
        └─ factorial(2, F2)
            └─ factorial(1, F3)
                └─ factorial(0, F4)
                    └─ F4 = 1   ← caso base
                F3 = 1 * 1 = 1
            F2 = 2 * 1 = 2
        F1 = 3 * 2 = 6
F = 4 * 6 = 24
```

Ejemplo Factorial con el caso Base como una Regla

```
% Caso base (como regla): el factorial de 0 es 1
factorial(N, 1) :- N =:= 0.

% Caso recursivo: N! = N * (N-1)!
factorial(N, F) :- N > 0, N1 is N - 1, factorial(N1, F1), F is N * F1.
```

Consulta

factorial(4, F)

Camino de Resolución

```
factorial(4, F)
    └─ factorial(3, F1)
        └─ factorial(2, F2)
            └─ factorial(1, F3)
                └─ factorial(0, F4)
                    └─ F4 = 1   ← caso base
                F3 = 1 * 1 = 1
            F2 = 2 * 1 = 2
        F1 = 3 * 2 = 6
F = 4 * 6 = 24
```

Intérpretes online útiles para testear código:

- https://swish.swi-prolog.org
- SWI-Tinker

Contenido

Paradigma Lógico ____ *1*
Cláusulas de Horn ____ 1
Tipos de Datos ____ *1*
Estructuras ____ 2
Listas ____ 2
Algunas Reglas Sintácticas ____ *2*
Operadores Aritméticos ____ *2*
Conceptos ____ *3*
Variable ____ 3
Unificación ____ 3
Cláusula ____ 5
Hecho ____ 5
Regla ____ 6
Consultas (queries) ____ 6
Functor ____ 7
Predicado ____ 7
Inversibilidad ____ 7
Pattern Matching ____ 8
Motor de inferencia ____ 8
Resolución SLD ____ 9
DFS (Depth First Search) ____ 9
Árbol SLD ____ 9
Backtracking (retroceso) ____ 9
Operador ! (Cut) ____ 10
Predicados ____ *11*
Predicados de Primer Orden ____ 11
Predicados de Orden Superior ____ 12
Recursividad ____ *13*